

ADT of Binary Tree

ADT BinaryTree

```
{  
    Objects:  
  
    A finite set of nodes either empty or consisting of a root  
    node, left BinaryTree and right BinaryTree.  
  
    Operations:  
  
    BinaryTree ();  
    // Creates an empty BinaryTree  
  
    BinaryTree (ElementType info);  
    // Creates a single node BinaryTree with information info  
  
    BinaryTree (BinaryTree lbt, ElementType info, BinaryTree  
    rbt);  
    // Creates a Binarytree whose left subtree is lbt, whose  
    right subtree is rbt,  
    // and whose root node contain info.  
  
    Boolean IsEmpty();  
    // If number of elements in the BinaryTree is 0 return TRUE,  
    // otherwise return FALSE.  
  
    BinaryTree LChild();  
    // If IsEmpty(), return error, else return left subtree  
  
    ElementType Data();  
    // If IsEmpty(), return error, else return data in the root  
    node  
  
    BinaryTree RChild();  
    // If IsEmpty(), return error, else return right subtree  
};
```

Static Array Representaion

The n nodes of an almost complete binary tree can be numbered from 1 to n , so that the number assigned to a left child is twice the number assigned to its father, and number assigned to a right child is one more than twice the number assigned to its father.

In C++, arrays start from 0; therefore, we will number the nodes from 0 to $n-1$. The left child of node at position p is at position $2p+1$ and right child is at $2p+2$. The root of the tree is at position 0.

```
# define NUMNODES 500

struct TreeNode
{
    int info;
    int left, right, father;
};

TreeNode BT[NUMNODES];
```

Exercise 8.1

Using the above node structure implement the Binary Tree ADT.